PRACTICAL 9-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 9 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Counter Race, Seat Booking Race, Parallel Sum and Part B: Runnable workers and synchronized account operations.

CODE SCREENSHOT:

CounterRace.java

Part A > Practical 9-A > CounterRace.java > ...

```java
public class CounterRace {
    static class Counter {
        int value = 0;

        synchronized void increment() {
            value++;
        }
    }

    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        Counter c = new Counter();
        Thread[] ts = new Thread[10];
        for (int i = 0; i < ts.length; i++) {
            ts[i] = new Thread(() -> {
                for (int j = 0; j < 1000; j++)
                    c.increment();
            });
            ts[i].start();
        }
        for (Thread t : ts)
            t.join();
        System.out.println("Final counter: " + c.value + " (expected 10000)");
    }
}

```

Code Screenshot 1

ParallelSum.java

Part A > Practical 9-A > ParallelSum.java > ...

```java
import java.util.*;

public class ParallelSum {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        int[] values = new int[10000];
        Arrays.fill(values, 1);
        long[] sums = new long[4];
        Thread[] ts = new Thread[4];
        for (int i = 0; i < 4; i++) {
            final int k = i;
            ts[i] = new Thread(() -> {
                for (int j = k * 2500; j < (k + 1) * 2500; j++)
                    sums[k] += values[j];
            });
            ts[i].start();
        }
        for (Thread t : ts)
            t.join();
        System.out.println("Parallel sum: " + Arrays.stream(sums).sum());
    }
}

```

Code Screenshot 2

SeatBooking.java

Part A > Practical 9-A > SeatBooking.java > ...

```java
public class SeatBooking {
    static class Cinema {
        int seatsLeft = 5;

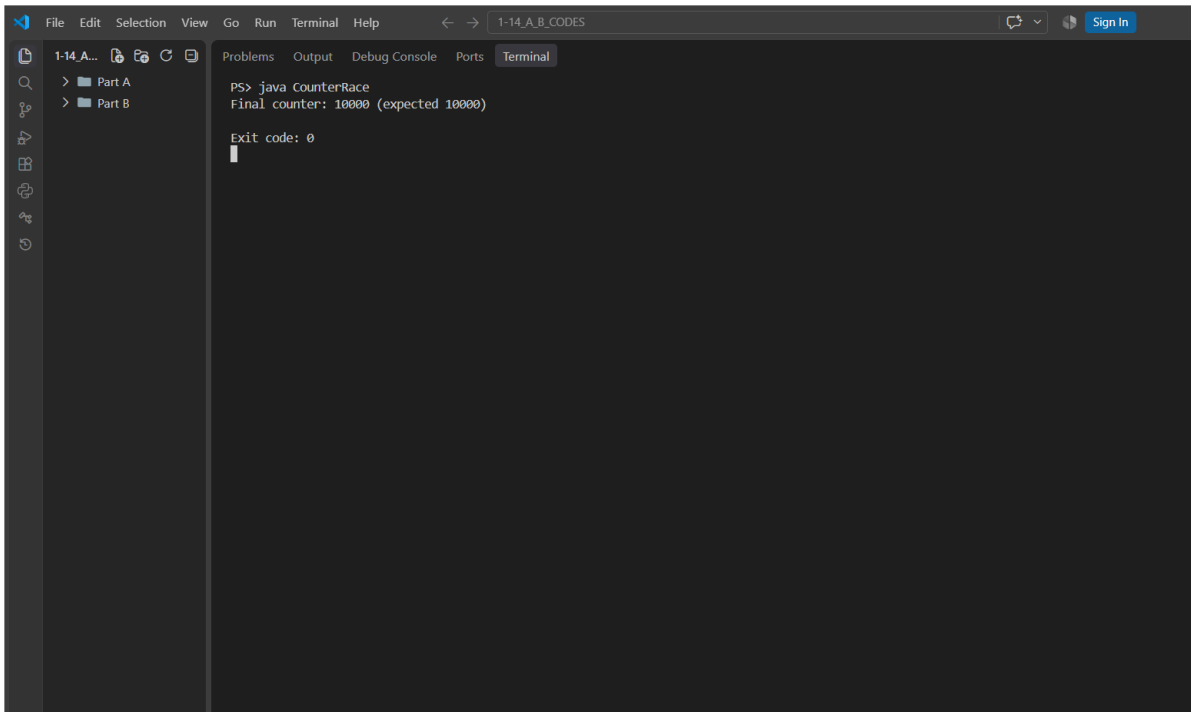
        synchronized boolean book() {
            if (seatsLeft > 0) {
                seatsLeft--;
                return true;
            }
            return false;
        }
    }

    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        Cinema c = new Cinema();
        int[] booked = { 0 };
        Thread[] ts = new Thread[10];
        for (int i = 0; i < 10; i++) {
            ts[i] = new Thread(() -> {
                if (c.book())
                    synchronized (booked) {
                        booked[0]++;
                    }
            });
            ts[i].start();
        }
        for (Thread t : ts)
            t.join();
        System.out.println("Booked: " + booked[0] + ", seats left: " + c.seatsLeft);
    }
}

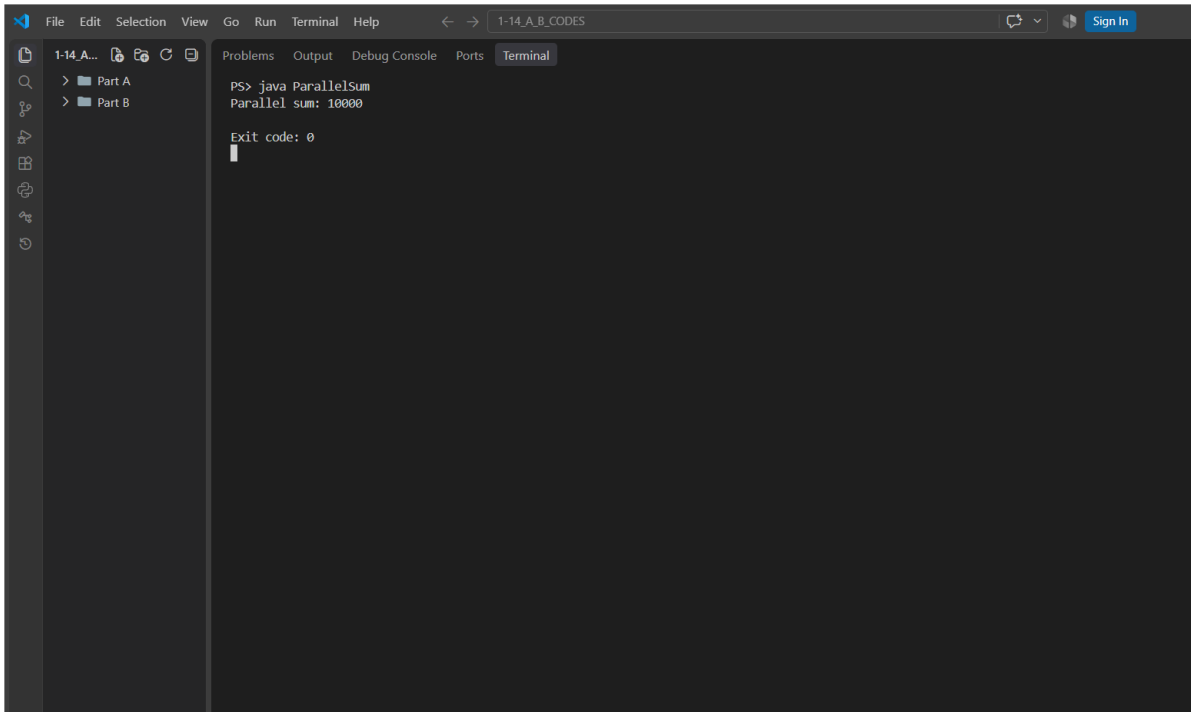
```

Code Screenshot 3

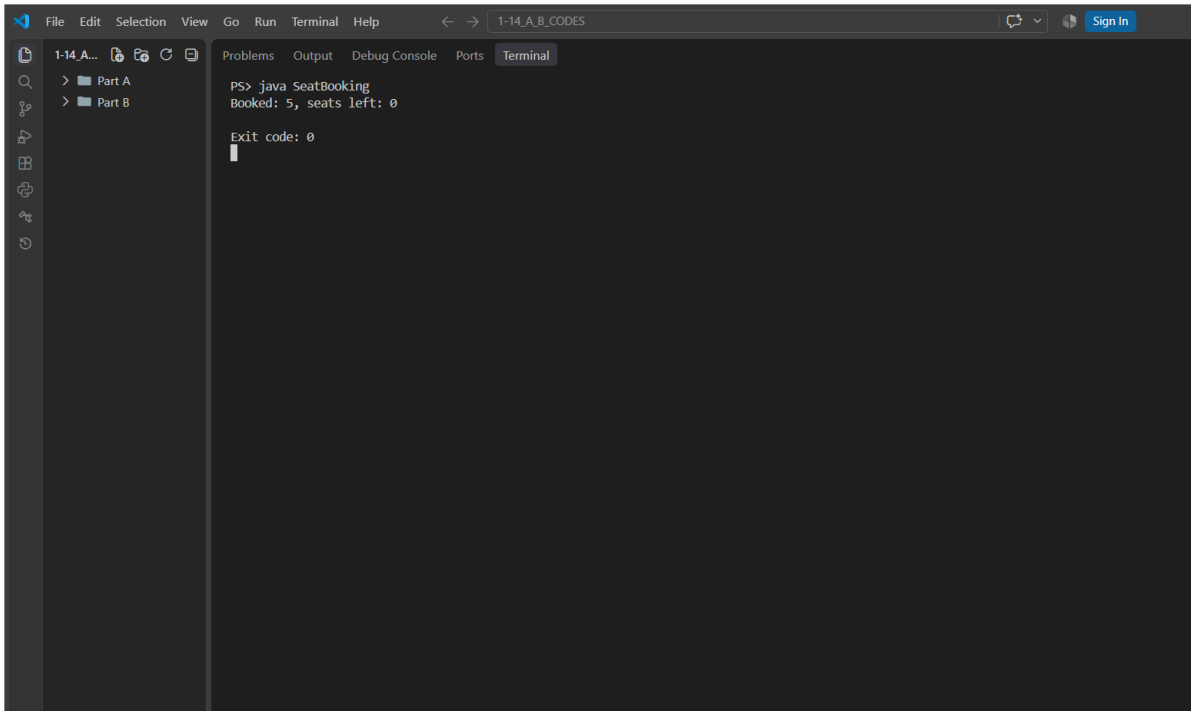
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Explain race conditions and protect shared mutable state with synchronization and thread coordination.